

How to Send Messages in Class: Sample Solutions

Year 11 Profile Course with Ms. Fernandez and Josh

November 29, 2017

1 Clarifications From the First Section

Many of you seemed to be confused as to whether one is only allowed to put one letter into our encrypt or decrypt function. In our first section we saw both:

First we saw an encryption of an alphabet as the encryption of individual letters from $A = \{a, b, c, d, e\}$ to $A' = \{d, e, a, b, c\}$. You might see that and say “hey wait, that’s just changing the order of the alphabet!” and you would be very correct! I would say, however, that my original notation wasn’t a very good one. Indeed the next thing I did was to shift the entire alphabet as a word:

Shift 1:

$$\text{encrypt}(abcdefghijklmnopqrstu\text{vwxyz}) \rightarrow bcdefghijklmnopqrstu\text{vwxyz}a$$

Shift 8

$$\text{encrypt}(abcdefghijklmnopqrstu\text{vwxyz}) \rightarrow ijklmnopqrstu\text{vwxyz}abcde\text{fgh}$$

This at least seems to fit our intuition more: if we are really “using” a cipher (“in real life”) then we would love to just be able to type something and have the computer spit out the right letters. In some sense, the computer *is* only encrypting one letter at a time, but that would make for very messy notation if to encrypt the word “albatross” we had to write the following:

$$\text{encrypt}(a)\text{encrypt}(l)\text{encrypt}(b)\text{encrypt}(a)\text{encrypt}(t)\text{encrypt}(r)\text{encrypt}(o)\text{encrypt}(s)\text{encrypt}(s)$$

instead of just

$$\text{encrypt}(\text{albatross})$$

But let me just say, the question was a very useful one! What you are allowed to put in a function is a super important and interesting question! For example, when we first learned subtraction as a little kid we may have been told that doing $5 - 4$ was fine but that $4 - 5$ wasn’t kosher. This is a lie of course, just perhaps a useful one for people who wanted to shelter us from the horrors of negative numbers.¹ The same is true of the square root function with negative numbers—before one has been exposed to complex numbers² putting a negative under a radical sign³ it was just a no go, then we extended the definition of the function to include negatives and $i = \sqrt{-1}$ and all that jazz. Wait, redefining functions? Yup! we can do that! there

¹PS: there is no need to do this, having searched, I can find no study that indicates it is in any way harmful to expose young children to negative numbers. Incidentally, the same is true of complex numbers, but that is a whole different story as they aren’t even really part of the curriculum any more in many school systems despite their extreme usefulness in a variety of applications.

²As mentioned in the footnote above, we will perhaps go into these eventually as they are great practice for various aspects of algebra and calculation that a lot of people find really tricky, but complex numbers make all of it so pretty!

³this symbol: $\sqrt{}$ also known as a “surd” to you British people, but what a word “surd” ... the word makes your face into the one you would get after eating a spoon full of Marmite...

is nothing “set in stone” about this stuff—well, there are conventions and things that we generally agree on (and learning them are indeed part of learning any field, one of the reasons mathematics is good practice) but we extend definitions all the time! okay, that is enough intro for now, let’s just go through the following problems.

Two things to keep in mind:

- These are just sample solutions, meaning if you have different answers they still might be totally correct and good and might in fact be more interesting or creative than the ones I have here—Mrs. Fernandez and I will be giving you feedback on your work, don’t worry.
- The notation I will use is what you might call “standard” computer science (Informatik) notation which can be a *little* different from pure math (for example, I often start indices at 0) but mostly it is the same, such that any person at all used to mathematical notational conventions (whether chemist, or physicist, etc.) could pick this up and basically say “oh yeah, I get what you are saying!” This level of clarity should basically be the goal of all of your answers: we don’t care what notation you use, so long as you explain it and make it clear to the reader. Things to avoid are pronouns, naming multiple variables the same thing, talking about “the alphabet” or “the word” when there might be more than one.

Super Important Sentences

Forget the math for a second, we don’t expect you immediately to change to this (significantly higher) level of solving and writing up problems. The struggle is real, but important, but even more important is recognizing the time that it takes to get used to new or different ways of thinking. Some of you are interested in going into fields that involve mathematical thinking, so you might say some of the applications are more obvious. But actually, we conceived of this as a way for people to get used to thinking and arguing in a clear and slightly more formal way than we might see in an English or German or History class. The skills we are working on here together are not just for school, and I know you hear this a lot, but writing will make up a significant portion of your life, and how you write—how clearly, how effectively, how compassionately, and how creatively, will directly impact how wonderful that portion of your life is. If you feel generally confident in your ability to understand this stuff, but have trouble writing stuff down in a way that is clear—welcome to the world of technical writing. Solving a problem takes a certain type of understanding, conveying that answer in a way that the reader doesn’t get lost is a whole different level. We are aiming for the latter, it will most likely take time, but that is what this year is about—the transition from calculation to deeper mathematical thinking, from answers that are numbers to answers that are paragraphs. It doesn’t get much more real than this!

2 Shift Cipher Problems

These problems assume that we have a shift cipher on our normal English alphabet (sorry German). Let’s start things slow and then get to juicier questions for our second question:

1. If our *key* (shift amount) is 6:

- (a) what are $encrypt(a)$, $encrypt(j)$, $encrypt(y)$?

Solution: We are essentially adding 6 to each letter here, so $encrypt(a) \rightarrow g$, $encrypt(j) \rightarrow p$, $encrypt(y) \rightarrow e$. Note on this last one we had to “wrap” back around to *a* and keep going from there.

- (b) What would the encrypted version of “albatross” and “sequester” look like?

Solution: There isn’t much different here except we have to go through the tedium of encrypting every letter: $encrypt(albatross) \rightarrow grhgzxuyy$ $encrypt(sequester) \rightarrow ykwakyzkx$ I am using this symbol $\rightarrow$ instead of $=$ just to show that we are really “mapping” something by putting something through this function. The equal sign just means too many different things!

- (c) What are $decrypt(abcd)$ and $decrypt(YOUR_NAME)$

Solution: $decrypt(abcd) \rightarrow uvwx$ this is just going backwards! Can you guess my middle name if this is what I got when I decrypted with the shift-6 cipher? *DimbouMugoyfUulih* Ha! just gave you another problem to do in the solutions! Take that genre constraints!

- (d) Is there a word that is made of up of all the same letters as its encrypted counterpart? How long can it be?

Solution: This question has prompted some really interesting solutions and some truly deep thinking. Let me give a couple solutions:

- Simpler but no less important: any word that contains all the letters of the alphabet has all the same letters as its encoded counterpart. The tricky thing here is that I actually already gave you all an example early on with our Shift-8 cipher:

$$encrypt(abcdefghijklmnopqrstuvwxyz) \rightarrow ijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyz$$

but we could just as easily put the alphabet in all jumbled up, or doubled:

$$encrypt(abcdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyz) \rightarrow ijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyz$$

It ain't pretty but it works! This means that you could actually have as long a word as you want! Maybe even infinite?!? ⁴ This could make you throw up your hands and say "well if there wasn't a largest one why did you ask us that question?!?! Well, because it shows that you understand you can put stuff longer than the alphabet in our encrypt function—which makes sense, as it would be less than wonderful if all messages had to be 26 characters long or less—what about Harry Potter shifted by 17? Sure! Go for it!

- The other solution that came up a couple times was to start with one letter and just keep encoding it until we either got through all of our letters or we got back to the letter we started on.

For example, if we start at a with a shift-6 cipher we would do the following:

$$\begin{aligned} encrypt(a) &= g \\ encrypt(encrypt(a)) &= encrypt(g) = m \\ encrypt(encrypt(encrypt(a))) &= encrypt^3(a) encrypt(m) = s \\ (yougettheidea...) &= encrypt^4(a) = encrypt(s) = y \\ &\dots eventually \dots \\ encrypt^{13}(a) &= encrypt(u) = a \end{aligned}$$

Whaaaaat?!?!?! Okay that is cool right? Somehow we magically get the word *agmsyekqwlioua*! This word is 13 letters long and only appears when we choose a shift value that is *even*. When our shift value is 5 then after repeating $encrypt^k(a)$ for k from 1 to 26 then we get the word: *afkpuzejotydxnrmrbglqva* which is 26 letters long and only repeats after going through each letter once! Now that is cool! What is it about 5 and 6 that make this big difference? Well, we have a hint in the sense that the 13 part only happens when it is even, and that means that if we have a shift of 13 then we just get back to a after two iterations, that is for Shift=13: $encrypt(encrypt(a)) = encrypt(n) = a$ which is also cool!

We will potentially come back to this in a later unit if we choose to tackle ideas surrounding "modular arithmetic" a field that is super interesting and you already have some intuition about.⁵ If you are interested in this stuff let us know and we can always either create or refer you to material on it!

⁴Whoa! Wait a second there! Infinite words are a bit tricky, we have to be careful about... you know what? yeah, why not? let's allow infinite words!

⁵Hint: it is how we measure time and basically what we have been doing since we started to play with ciphers.

- (e) For all the words you encrypted in the previous questions, see if you can encrypt them again with 20 as your key, what happens? What pattern do you notice? What would the equivalent be if our key was 17, or if it was 13? (You don't necessarily need to do all of these over again of course, just demonstrate that you understand what is going on.) How might we write the relationship between a 6-shift code and a 20 shift code?

Solution: This is hinting at the idea that a cipher with shift 20 is kind of like a cipher with shift -6, or shifting *to the left* instead of the right. When I say “kind of like” here, what I mean is, “totally the same thing.” Here the letter a shifted (to the right) by 20 we get u , and when it is shifted (to the left) by 6, which we might as well write as “-6,” we also get u . Why is this? Well, because $20 + 6 = 26$ right? So that is a decent answer, but the key is to understand the idea that on our normal 26 letter alphabet for any cipher with shift k the cipher $-k$ is *equivalent to*⁶ as the cipher $26 - k$. This works independent of what alphabet we are using!

Suppose our alphabet is the list of characters A and we call the size of our alphabet $|A|$ then a shift cipher on this alphabet with shift $-k$ (for $k < |A|$) is equivalent to the cipher with shift $(|A| - k)$. To show this formally has a couple annoying bits (because we would need to show that these k s are unique etc.) but that is beyond our scope here.⁷

2. Now let's switch to a more general key k , so we don't specifically how much we are shifting by, perhaps we can still say some cool stuff about these ciphers—feel free to use examples:

- (a) A *palindrome* is a word that is the same backwards and forwards (like mom, kayak, racecar, or deified), are palindromes still palindromes after we encrypt them with a shift cipher? Can you prove it? What if we use some other type of cipher on the same alphabet? What notation might you use to prove it with fewer words or symbols? (Be sure to clearly define what you mean by each symbol or abbreviation!)

Solution: So first of all, I would start with the next problem and then come back to this one—this is neither illegal nor restricted, in fact, we highly encourage that you look over all problems before doing them and skip ones you aren't sure about to go on to tackle later ones.⁸ So what I would do here is use a *reverse*($\cdot$) function, a function where inputting the word $abcde$ would return $edcba$, as described in the next problem.

The question of whether a palindrome is also a palindrome after it has been encoded by our cipher is the same as saying the following: if $w_k = w_{-k}$ is *encrypt*(w_k) *also equal to* *encrypt*(w_{-k})? That is the key question! But we know the answer to that—we can just replace w_{-k} with w_k in our equation and we get *encrypt*(w_{-k}) = *encrypt*(w_k) which is exactly what we wanted! Since this works for all k , if you aren't convinced it does, check out the function in the next question, we can be sure that a palindrome will remain a palindrome after being encoded!

- (b) In order to do the previous problem, you probably had to invent some type of notation that conveys the reverse of a given word: how would you write a function *reverse*($\cdot$) in terms of the letters in a given word $w = w_0w_1 \cdots w_{n-1}w_n$? What happens if we do *reverse*(*reverse*(w)))? With that in mind, what can you say about the inverse of *reverse*($\cdot$)?

Solution: Here is how I would write this function *reverse* acting on a word w :

Prerequisites: Notice that the “middle” of this word is the empty space between the $\frac{n}{2}$ 'th and $\frac{n+2}{2}$ 'th characters (written $w_{\frac{n}{2}}$ and $w_{\frac{n+2}{2}}$ respectively if n is even and is exactly the $\lfloor \frac{n+1}{2} \rfloor$ 'th character or $w_{\lfloor \frac{n+1}{2} \rfloor}$ when n is odd. We will be using this $\lfloor \frac{n+1}{2} \rfloor$ in our function.

⁶fancy mathematical term meaning “the same as”

⁷As in the previous problem this gets to the core of the ideas of Modular Arithmetic—talk about sneaky! We started teaching you guys Modular Arithmetic and cryptography and even some basic group theory, which are totally not in the curriculum, just through trying to introduce the concept of functions! Geez, we are really intellectually abusing you all! It's just awful!

⁸A city you called home will always show its slight differences, offer you only a near perfect facsimile of its past self, your eye will see new details while reliving old experiences, a problem is such a city, made up of the same buildings and structures, you can always go back, but your view is forever changed, transformed by the other cities you visited since making the leap and leaving.

Also remember that $-k = n - k$ for n , the length of our word.

Function *reverse*(word w):

For all integers k from 0 to $\lfloor \frac{n+1}{2} \rfloor$ **do**:

 Swap w_k and w_{-k} .⁹

Return: w (Note, now it is a modified w !)

- (c) If I give you access to an encryption machine, and tell you that it is a Shift Cipher, how many times must you use the machine in order to guess which shift I am using? What method would you use? Suppose I change the alphabet of the machine, what would you need besides just the new symbols I am using in order to figure out my shift cipher in the same amount of steps?

Solution: You only need 1 symbol! If I give the machine a then I immediately know that the shift value is: Here is how you would find it.

First: Assign a number from 0 to 25 to each of the letters in a nice table (in order obviously, no fancy tricks here!). Each letter has an associated number so that $number(b) = 1$ and $number(u) = 20$. Get it?

Second: Put the letter a in the machine to get $encrypt(a)$.

Third: Look up $encrypt(a)$ in our table, it will have some number, we might call it $number(encrypt(a))$, but that number is our shift value! and we are done!¹⁰

- (d) Schlomo has solved the previous problem and is rather unhappy with his result, he wants to make a given Shift Cipher more “secure” by shifting each letter twice, first by k_1 then again by k_2 . Does this make the cipher more secure? Why or why not? As a hint, it might be helpful to think of $encrypt(\cdot)$ as a *multi-input function*, that is, a function that takes in more than one input (just like it says on the tin!). So instead of $encrypt(w)$ with an “implicit” key k for a given word w , we can try something like $encrypt(w, k)$. How might one write Schlomo’s strategy with this new(ish) function? Can you think of another strategy involving manipulating keys that makes this “more secure” (in your answer, be sure to specify what you mean by more secure).¹¹

Solution: Ah Schlomo, so well meaning, but so misguided. The answer is unfortunately, no, this multiple encryption scheme doesn’t make our cipher any safer... :(We can take the first function $encrypt(w, k_1)$ and then his second $encrypt(w, k_2)$ and if we do one after the other we just get $encrypt(w, k_1 + k_2)$... but this is just another shift cipher with a new key that we can call $q = k_1 + k_2$. So just like we saw in part (c) we know we can crack his code with just one letter. Note that if q is greater than 26 then we just keep subtracting 26 until we get to something less than 26—In other words, for every q we could always add 26 to q and end up with an equivalent cipher!¹² Interestingly enough, we can’t recover each individual key k_1 and k_2 using part (c)’s method, in fact—there is no way to do that for sure without more information¹³but we can most certainly decrypt any message so long as we have our new key (q) and that is unfortunately enough to foil Schlomo’s plan.

⁹This isn’t the clearest it could be, what we mean by swap is store w_k somewhere, replace w_k with the value of w_{-k} and then replace w_{-k} with our stored w_k . That is how our swap works!

¹⁰If you find this to be less than magical, think about the magical fact that Sotolon tastes like maple syrup in low concentrations and like curry in high concentrations. Think about adding a little of something to your pancakes and then adding too much to have it taste like curry or fenugreek! Now let’s move on.

¹¹by the way, what we just did is to “uncurry” our encrypt function: currying is where we use multiple one-argument/input functions to describe a multi-argument/input function, uncurrying is the reverse! Can you think of other examples where this might be useful? If you are interested in this, bug us about it! (But also look it up it’s super cool!)

¹²That gosh darn Modular Arithmetic again!

¹³This is the property used by something confusingly called “zero knowledge proofs” which are definitely worth a Wikipedia adventure!